

LPFT / Agentic Finance Exchange

Analisi tecnica completa ed estesa — stato del progetto

Data report: 18 May 2026

Versione: estesa con diagrammi d'architettura

Indice

- 0 Architettura visuale — diagrammi
- 1 Executive summary e visione prodotto
- 2 Architettura logica e flussi dati
- 3 Stack tecnologico dettagliato
- 4 Agente fiduciario (chat AI)
- 5 Tool chat — specifica funzionale
- 6 Motore quant TypeScript (event-driven)
- 7 Libreria lib/ (moduli applicativi)
- 8 API REST Next.js — inventario
- 9 API FastAPI Python — inventario
- 10 Schema Prisma — modelli e migrazioni
- 11 UI React — componenti principali
- 12 Metriche e interpretazione
- 13 RLFF e miglioramento modello
- 14 Exchange, routing e on-chain
- 15 Fasi di implementazione A–D
- 16 Test, qualità e debito tecnico
- 17 Operatività e troubleshooting
- 18 Variabili d'ambiente
- 19 Appendice — struttura directory
- 20 Guardrail esecuzione e routing
- 21 Motore Python lpft_shared
- 22 Script operativi
- 23 Compiler strategia quant
- 24 Stato repository Git
- 25 Glossario
- 26 Conclusioni e prossimi passi

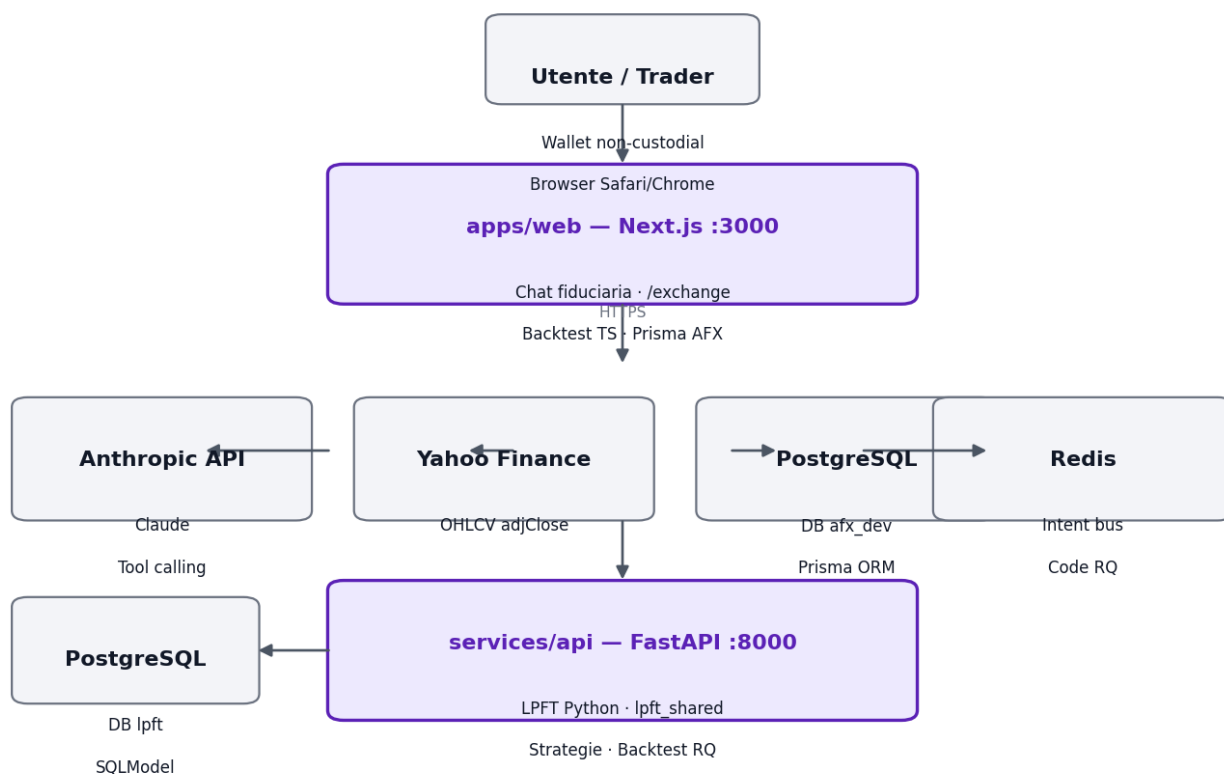
0. Architettura visuale — diagrammi

Questa sezione riassume il progetto in modo visivo. Ogni figura corrisponde a un file PNG in docs/report-assets/ (rigenerabile con `technical_report_diagrams.py`). Leggere le figure prima dei capitoli testuali aiuta a collocare LPFT (ricerca Python) e AFX (exchange + chat Next.js) nello stesso prodotto.

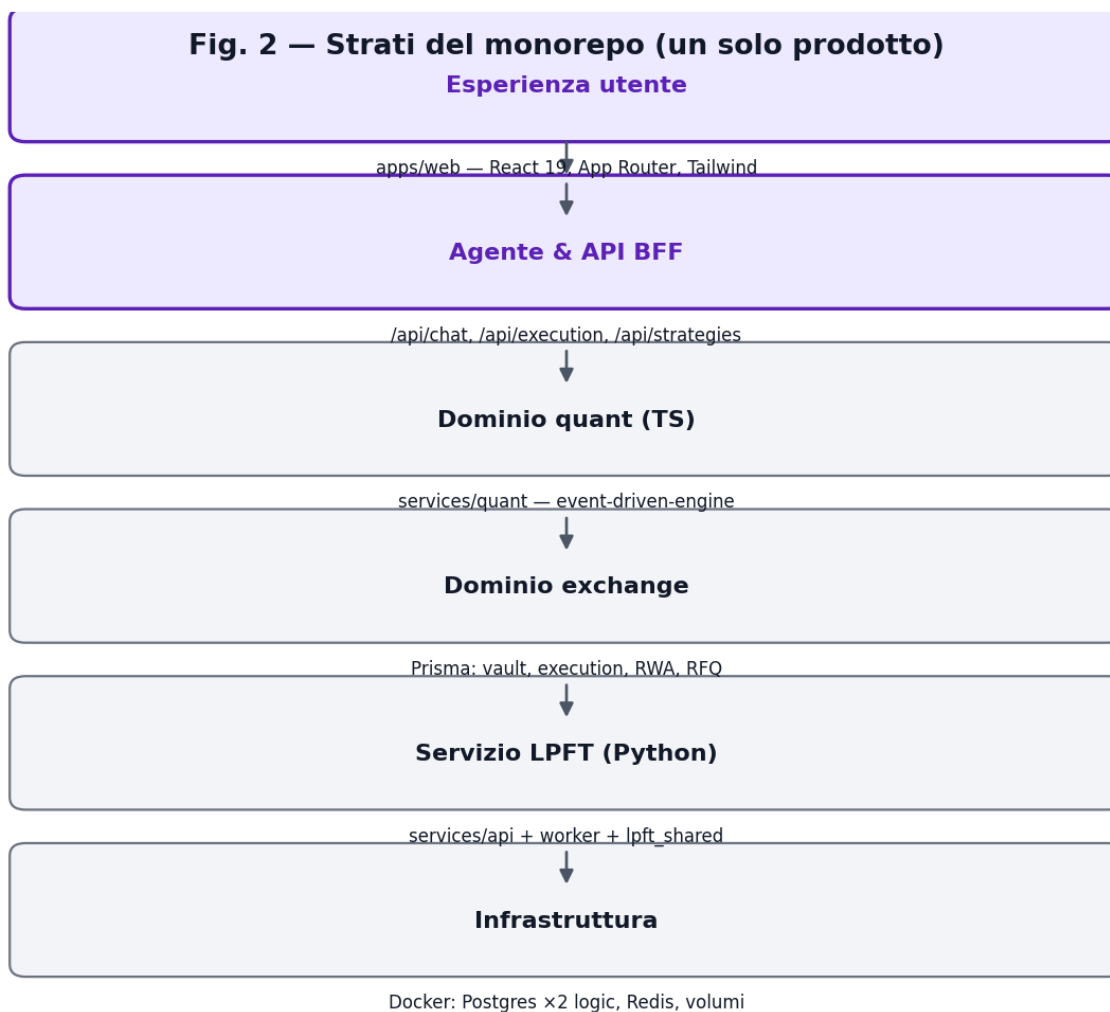
0.1 Cosa fa il prodotto in una frase

Un assistente quantitativo istituzionale che conversa in italiano, simula strategie su dati di mercato reali, produce report analitici con grafici e trade, e — su approvazione dell'utente — propone esecuzioni tracciate per audit e miglioramento del modello (RLFF), con scaffold verso vault non-custodial on-chain.

Fig. 1 — Contesto di sistema (vista d'insieme)



Contesto di sistema: utente, Next.js, API Python, servizi esterni.



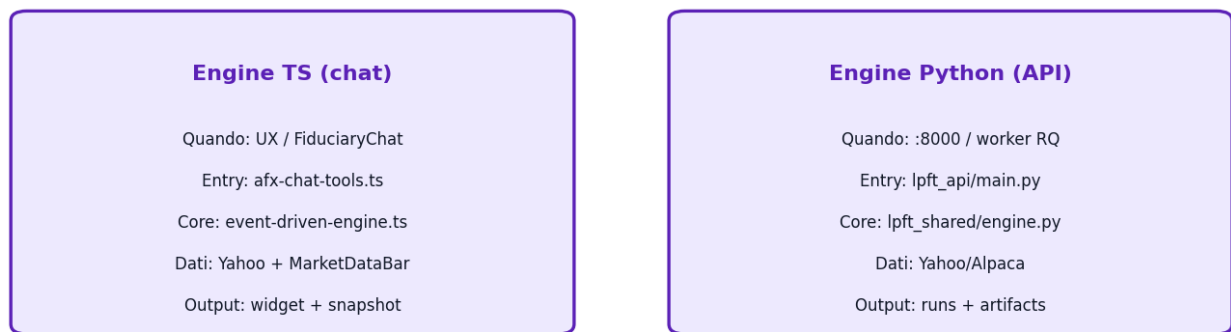
Strati del monorepo dal UI all'infrastruttura.

Fig. 3 — Flusso chat → backtest → report (percorso principale)

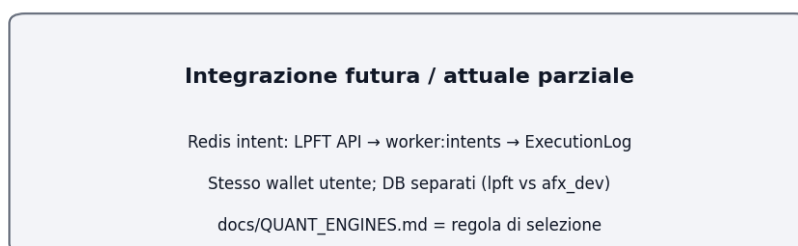


Flusso principale dalla chat al report salvato.

Fig. 4 — Due motori quant (coesistenza documentata)

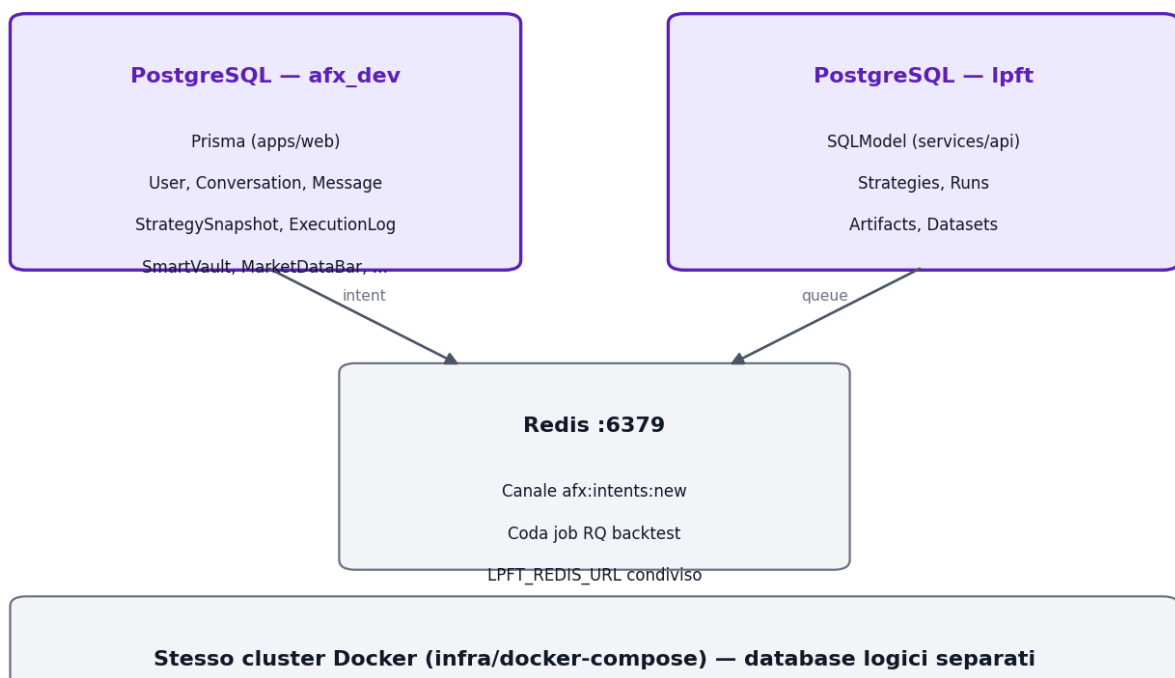


NON merge in Fase A — scelta per contesto



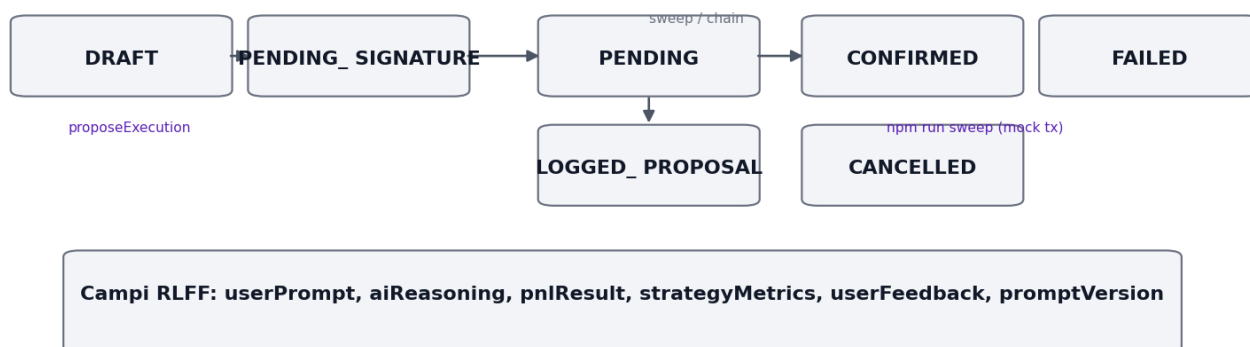
Confronto motori TypeScript e Python.

Fig. 5 — Topologia dati e persistenza



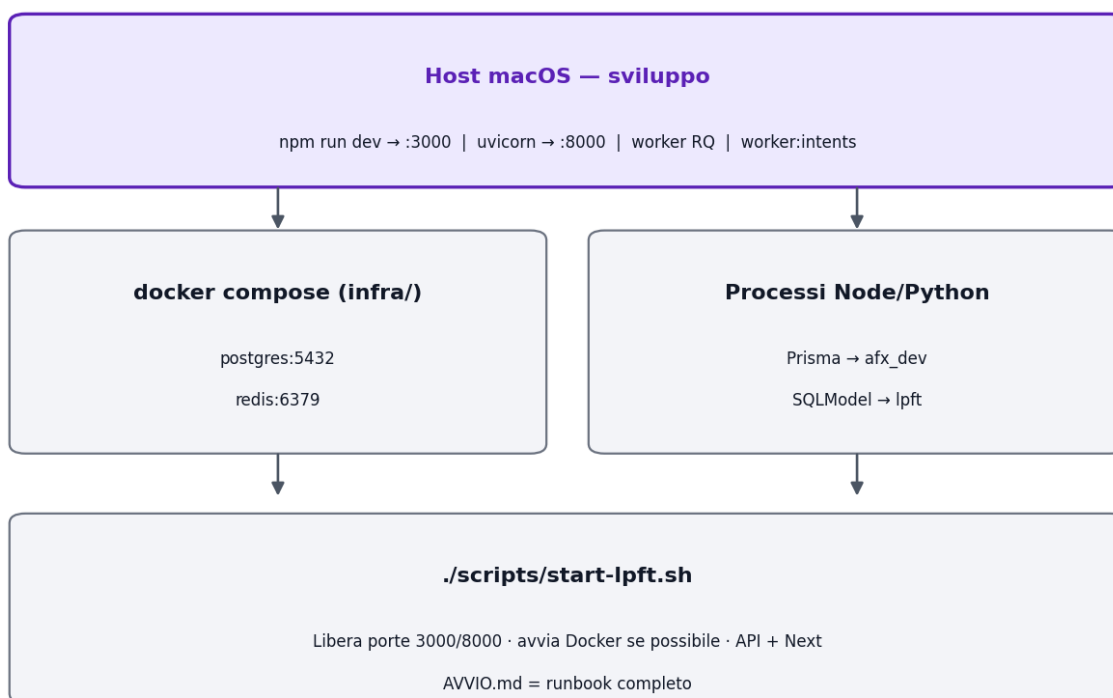
Database afx_dev, lpft e Redis.

Fig. 6 — Ciclo di vita ExecutionLog (esecuzione / RLFF)

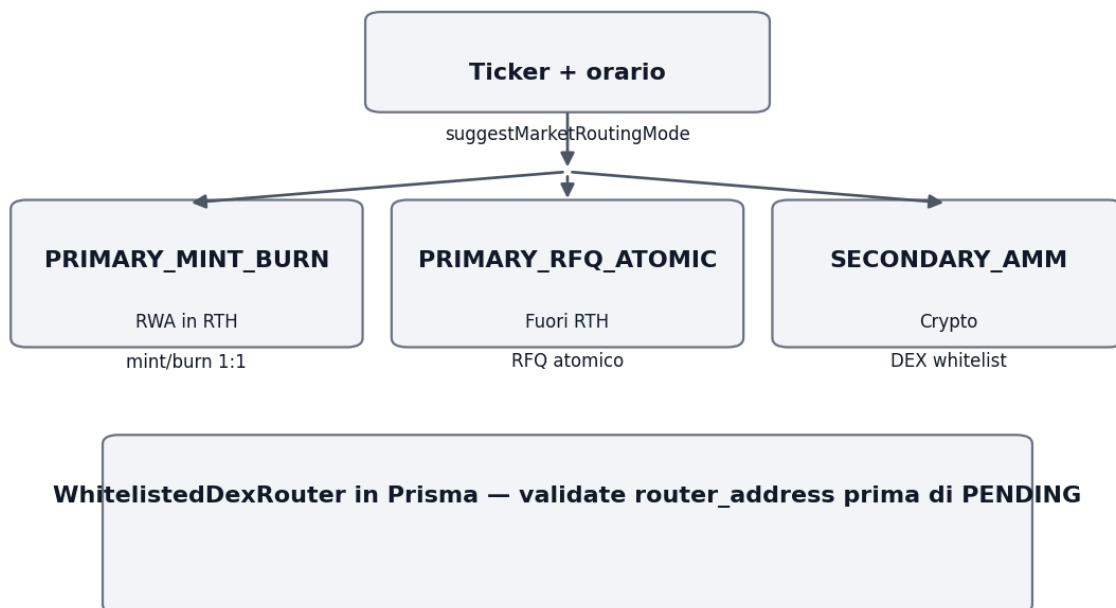


Stati ExecutionLog e RLFF.

Fig. 7 — Infrastruttura locale (Docker + processi host)



Deploy locale Docker + processi.

Fig. 8 — Routing mercato (PRIMARY vs SECONDARY)

Decisione routing mercato primario/secondario.

1. Executive summary e visione prodotto

LPFT (Laboratory for Programmable Finance & Trading) e AFX (Agentic Finance Exchange) convergono in un unico prodotto: un exchange finanziario agentic non-custodial. L'utente è OWNER dello SmartVault on-chain; il backend agisce come MANAGER con permessi limitati (trade verso router whitelisted, nessun prelievo arbitrario). La ricerca quantitativa (backtest, metriche, spiegazione) avviene in chat con Claude; l'esecuzione reale passa da proposeExecution → firma → chain (oggi in gran parte mock/scaffold).

Principio architetturale (docs/UNIFIED_EXCHANGE.md): non due prodotti concorrenti, ma strati complementari — ricerca/intent (LPFT), exchange/audit (AFX), infra conmotione (Postgres doppio, Redis). La cartella agentic-finance-exchange/ è archivio storico; l'unica app UI è apps/web sulla porta 3000.

2. Architettura logica e flussi dati

Le Fig. 1–3 mostrano il contesto e il percorso felice dell'utente. La Fig. 4 chiarisce perché esistono due motori di backtest. La Fig. 5–7 descrivono dove finiscono i dati e come si avvia l'ambiente locale. La Fig. 8 illustra come si sceglie il mercato primario vs secondario prima di un'operazione.

2.0 Ruoli e confini di responsabilità

- Utente (OWNER): possiede il wallet e autorizza; non delega la custodia dei fondi al backend.
- Agente IA (MANAGER logico): propone strategie e ordini; non esegue trade senza conferma UI.
- apps/web: unica interfaccia prodotto; traduce intent in JSON, backtest, snapshot, widget.
- services/api: motore ricerca legacy, dataset, generazione programma; opzionale per la chat.
- Prisma (afx_dev): verità per chat, execution log, strategie salvate, cache OHLCV.
- SQLModel (lpft): verità per run storici API Python e artifact su disco/DB.

2.1 Flusso principale (chat → report)

- Utente invia messaggio su / con wallet (Zustand + default env).
- POST /api/chat: sanitize messaggi, getOrCreateUserByWallet, streamText Anthropic.
- System prompt: AFX_FIDUCIARY_SYSTEM (versione afx-fiduciary-v1).
- Tool disponibili: analyzeMarketData, buildQuantitativeStrategy, runStrategyBacktest, proposeExecution, executeTrade,

executeSwap.

- buildQuantitativeStrategy: compiler JSON → EventDrivenConfig → runEventDrivenBacktest → persistStrategySnapshot.
- Risposta UI: testo stream + widget (AnalysisInlineWidget, QuantStrategyWidget, ProposeExecutionWidget).
- Report persistente: /analysis/[id] con StrategyReportView (KPI, grafici, registro trade).
- Salvataggio esplicito: POST /api/strategies/[id]/save imposta savedAt → visibile in /strategies.

2.2 Flusso parallelo (API Python :8000)

- Client legacy o integrazioni chiamano FastAPI su LPFT_DATABASE_URL (db lpft).
- /generate-strategy, /generate-and-backtest: LLM + lpft_shared engine.
- Backtest in coda RQ se Redis+worker attivi; altrimenti inline nell'API.
- Con LPFT_AFX_INTENTS_ENABLED: intent_publisher scrive su Redis channel afx: intents: new.
- npm run worker: intents: valida router whitelist, crea ExecutionLog PENDING o LOGGED_PROPOSAL.

2.3 Diagramma testuale

```
[Browser] → :3000 Next.js → /api/chat → [Anthropic + Tools] → [TS Quant Engine] → Prisma afx_dev
      ↓               ↓
    /analysis/[id]   MarketDataBar (cache Yahoo)
[Browser/API] → :8000 FastAPI → lpft_shared → Postgres lpft + Redis RQ → Worker
```

3. Stack tecnologico dettagliato

3.1 apps/web — dipendenze runtime

- next ^15.2, react ^19, typescript ^5.7
- ai ^6.0 + @ai-sdk/anthropic ^3.0 + @ai-sdk/react ^3.0
- @prisma/client ^6.19, zod ^4.4
- lightweight-charts ^5.2, yahoo-finance2 ^3.14
- redis ^4.7, zustand ^5.0
- vitest ^3.2 (dev), eslint 9, tailwind 3.4

3.2 services/api — Python

- FastAPI, SQLAlchemy, Anthropic SDK, RQ, Redis
- lpft_shared: engine.py (program backtest), market_data.py (Yahoo/Alpaca, mock oracoli)
- Moduli: assistant.py, program_llm.py, strategy_spec_tool.py, intent_publisher.py, dsl.py

4. Agente fiduciario (chat AI)

4.1 Prompt di sistema (afx-fiduciary-prompt.ts)

Versione tracciata: AFX_PROMPT_VERSION = afx-fiduciary-v1. Il modello risponde in italiano, tono istituzionale, senza markdown pesante in chat. Deve spiegare prima/dopo i tool. Non invoca tool per domande generiche. Chiede chiarimenti se mancano asset/orizzonte/rischio.

4.2 Regole tool (estratto)

- analyzeMarketData: obbligatorio prima di proposte; OHLCV Yahoo reale.
- buildQuantitativeStrategy: strategia JSON completa + risk management obbligatorio.
- runStrategyBacktest: backtest semplice (buy_and_hold, drawdown_to_stable).
- proposeExecution: solo dopo backtest; guardrail Sharpe; crea ExecutionLog DRAFT.
- executeTrade / executeSwap: solo dopo conferma utente (non autonomi dall'agente).

4.3 Persistenza conversazione

POST /api/chat accetta conversationId opzionale. Se presente, salva Message user/assistant su Prisma (Conversation, Message). Titolo conversazione da primo messaggio utente (max 120 char).

5. Tool chat — specifica funzionale

5.1 analyzeMarketData

Input: ticker, timeframe (1y|2y|5y). Output: ultimo close, n barre, sample code. Usa fetchHistoricalOhlcv.

5.2 buildQuantitativeStrategy

Input: schema Zod buildQuantitativeStrategySchema (intentClass WALLET_MANAGEMENT | ALGORITHMIC_TRADING, symbol, benchmark, legs, riskManagement, rebalance, asymmetric weights, ecc.). Pipeline: validateQuantStrategyInput → compileToEventDrivenConfig → fetchUniversePriceMatrix → runEventDrivenBacktest → projectForwardFromCloses (30/90/365) → persistStrategySnapshot. Restituisce: metrics, series completa (dal giorno 0), trades, snapshotId, reportUrl, widget quant_strategy_v1.

5.3 runStrategyBacktest

Backtest classico su coppia allineata asset/benchmark. Strategie: buy_and_hold, drawdown_to_stable (maxDrawdownFrac, reentrySmaDays). Output widget con serie e metriche.

5.4 proposeExecution

Valida metriche con validateProposeExecution (Sharpe minimo, drawdown). Crea ExecutionLog DRAFT con idempotencyKey UUID, payload proposal_v2, strategyMetrics JSON. Collega snapshot report. Widget propose_execution_v1. Guardrail può rifiutare con rejected:true.

5.5 executeTrade / executeSwap

Recuperano ExecutionLog, invocano signer (mock KMS), aggiornano stato esecuzione. Integrazione on-chain reale ancora scaffold (calldataDigest, transactionHash mock nel sweep).

6. Motore quant TypeScript (event-driven)

6.1 Moduli in services/quant/

- event-driven-engine.ts: Loop giornaliero: segnali → pending rebalance T+1 → risk halt → mark-to-market → serie equity/benchmark.
- signal-engine.ts: Pesi target: equal weight, momentum, RSI, asymmetric multi-asset, session mask RTH.
- risk-manager.ts: Max drawdown portafoglio, stop loss per posizione, trailing stop, liquidazione forzata.
- execution-engine.ts: Rebalance con slippage/spread; close position; forced liquidation.
- portfolio-state.ts: Cash, posizioni, HWM, halt mensile reset, markToMarket.
- trading-friction.ts: Slippage bps per symbol class (crypto vs equity).
- trade-journal.ts: Registro trade simulati: entry/exit, pnlFrac, pnlEquity, reason codes.
- metrics.ts: CAGR, Sharpe, max drawdown da serie equity; annualizzazione 252 gg.
- backtest.ts: Orchestrazione: runEventDrivenBacktest, runStrategyBacktest, projectForwardFromCloses.
- strategy-adapter.ts: Adattatori spec legacy verso config event-driven.

6.2 Realismo e test quant

- tier1-realism.test.ts — vincoli realismo ordini
- trading-friction.test.ts — slippage e costi
- halt-circuit.test.ts — circuit breaker drawdown
- monthly-halt-reset.test.ts — reset halt mensile
- signal-engine.asymmetric.test.ts — pesi asimmetrici multi-asset
- event-driven-engine.test.ts — integrazione motore
- portfolio-state.halt.test.ts, risk-manager.test.ts

6.3 Dati di mercato (services/market_data.ts)

fetchHistoricalOhlcv, fetchPairedHistory, fetchUniversePriceMatrix da Yahoo. Persistenza opzionale su MarketDataBar (symbol, date, adjClose, volume). price_matrix.ts: allineamento calendario multi-ticker, maschere sessione regolamentata US.

7. Libreria lib/ (moduli applicativi)

- afx-chat-tools.ts — definizione tool AI
- afx-quant-compiler.ts — JSON strategia → engine config
- afx-quant-strategy-schema.ts — Zod input strategia
- afx-risk-caps.ts — cap drawdown/SL da input

- `afx-execution-guard.ts` — validazione `proposeExecution`
- `afx-snapshot-store.ts` — CRUD `StrategySnapshot`
- `afx-derived-stats.ts` — `totalReturn`, vol annua da serie
- `afx-market-routing.ts` — PRIMARY_* vs SECONDARY_AMM
- `afx-fiduciary-prompt.ts` — system prompt
- `afx-user.ts` — `getOrCreateUserByWallet`
- `afx-store.ts` — Zustand wallet/UI
- `series-analytics.ts` — drawdown series, monthly returns, advanced metrics
- `trade-stats.ts` — `summarizeTrades`, `yearlyReturns`
- `extractLatestAnalysis.ts` — parsing messaggi chat per widget
- `sanitize-chat-messages.ts` — pulizia payload UI
- `services/signer.ts` — KMS mock / interfaccia firma
- `api.ts` — client verso LPFT API :8000
- `backtestMetrics.ts` — etichette metriche report IT

8. API REST Next.js — inventario completo

- POST `/api/chat` — Chat streaming Anthropic + tool + persistenza messaggi
- GET|POST `/api/quant/backtest` — Backtest TS diretto (non chat)
- GET `/api/market/history` — Storico OHLCV
- GET `/api/market/quotes` — Quote mercato
- GET|POST `/api/analysis/snapshots` — Lista/crea snapshot
- GET `/api/analysis/snapshots/[id]` — Dettaglio snapshot
- GET `/api/strategies` — Strategie salvate (`savedAt` not null)
- POST `/api/strategies/[id]/save` — Bookmark strategia + titolo
- POST `/api/save-strategy` — Salvataggio alternativo
- GET `/api/execution/[id]` — Stato execution log
- POST `/api/execution/[id]/execute` — Submit firma/esecuzione
- POST `/api/execution/[id]/feedback` — RLFF user feedback
- GET `/api/vault` — Info vault utente
- POST `/api/vault/deploy` — Deploy vault (scaffold)
- GET `/api/health` — Health generico
- GET `/api/afx-health` — Health AFX/Prisma/dettaglio

9. API FastAPI Python — inventario

- GET `/`, GET `/demo` — Root e demo
- POST|GET `/strategies` — CRUD strategie LPFT
- POST|GET `/runs`, GET `/runs/{id}` — Esecuzioni backtest
- POST `/generate-strategy` — Generazione strategia LLM
- POST `/generate-strategy-stream` — Stream generazione
- POST `/generate-and-backtest` — One-shot
- POST `/generate-program` — Program DSL
- POST `/assistant/stream` — Assistant streaming
- POST `/runs/program` — Run programma
- GET `/runs/{id}/artifacts` — Artifact backtest
- POST `/datasets/upload`, `/datasets/fetch` — Dataset

10. Schema Prisma — modelli e scopo

- User: `walletAddress` unique; hub per vault, chat, execution, snapshot.
- StrategySnapshot: Report backtest: `equitySeries`, `trades`, `metrics`, `compiledStrategy`, `savedAt`/title.
- Conversation / Message: Storico chat; role user|assistant|system.
- WhitelistedDexRouter: Router DEX per chain; UNISWAP_V3/V2/CURVE.
- VaultFactoryConfig: Factory deploy vault per chainId.
- SmartVault: Istanza vault utente; status PENDING_DEPLOY|ACTIVE|PAUSED.
- RwaToken: Token sintetici RWA (bAAPL); `primaryWindowOnly`.

- RfqQuote: Quote RFQ primario fuori RTH; settlement off-chain.
- ExecutionLog: Core RLFF: prompt, reasoning, pnlResult, strategyMetrics, executionStatus, feedback.
- MarketDataBar: Cache OHLCV Yahoo per replay backtest.

10.1 ExecutionStatus — ciclo di vita

DRAFT → PENDING_SIGNATURE → PENDING → CONFIRMED | FAILED | CANCELLED. Anche: LOGGED_PROPOSAL, AWAITING_USER_SIGNATURE, AWAITING_MANAGER_SIGNATURE, SUBMITTED. Sweeper (npm run sweep) promuove PENDING con tx mock.

10.2 Migrazioni Prisma (ordine)

- 20260215120000_pending_idempotency
- 20260514120000_init_afx
- 20260515120000_execution_rlff_fields
- 20260516100000_pending_signature_status
- 20260517120000_strategy_snapshots
- 20260517140000_market_data_bars
- 20260518111000_phase_d_rlff_feedback_prompt_version
- 20260518120000_strategy_saved_bookmark

11. UI React — componenti principali

- FiduciaryChat.tsx — Chat principale, useChat, rendering widget
- ChatMessageBubble.tsx — Bubble + AnalysisInlineWidget
- AnalysisInlineWidget.tsx — Metriche + grafico + Salva strategia
- BacktestWidget.tsx — Grafico lightweight-charts equity
- StrategyReportView.tsx — Report completo multi-sezione
- TradeAnalysisBlock / TradeRegistryTable — Analisi trade in panoramica
- ProjectionOutlook.tsx — Proiezioni 30/90/365 gg
- ProposeExecutionWidget.tsx — Conferma esecuzione
- ExecutionStatusBar.tsx — Stato pipeline execution
- SaveStrategyActions.tsx — CTA salva / link report
- StrategyBuilder.tsx — UI costruzione strategia (legacy/demo)
- Exchange* — Pagine exchange mercati

12. Metriche e interpretazione

12.1 Metriche portafoglio

- CAGR: rendimento annualizzato composto da serie equity.
- Sharpe: excess return / volatilità annualizzata (rf≈0).
- Max drawdown: massimo calo da picco equity.
- Rend. periodo (widget): (equity_finale/equity_inizio)-1 su tutto il backtest.
- Alpha vs benchmark: rendimento strategia meno buy&hold benchmark.

12.2 Metriche trade

PnL % sul trade = movimento prezzo sul titolo (non rendimento portafoglio). Win rate, profit factor, hold medio da summarizeTrades(). Registro trade in report: entry/exit, pnlFrac, pnlEquity, reasonEntry→reasonExit.

13. RLFF e miglioramento modello

Ogni ExecutionLog conserva userPrompt, aiReasoning, pnlResult, strategyMetrics, modelId, promptVersion. POST feedback aggiorna userFeedback e feedbackAt. Script npm run rlff:export esporta dataset per training/fine-tuning. Obiettivo: reinforcement learning from human/analitics feedback su proposte reali vs simulate.

14. Exchange, routing e on-chain (scaffold)

14.1 MarketRoutingMode

- PRIMARY_MINT_BURN — RWA in regular trading hours; mint/burn 1:1.
- PRIMARY_RFQ_ATOMIC — Fuori RTH; RFQ atomico off-chain.
- SECONDARY_AMM — Crypto e mercato secondario; router whitelisted.

14.2 Signer e vault

lib/services/signer.ts: interfaccia KeyManagementService; createMockKmsSigner per dev. Env: AFX_SIGNER_MODE, AFX_KMS_KEY_REF. Vault deploy API stub. Mainnet checklist in UNIFIED_EXCHANGE.md: chain config, RPC confirmation, rate limit, audit.

15. Fasi di implementazione (roadmap eseguita)

Fase A — Consolidamento

- Persistenza conversazione Prisma; cache MarketDataBar; docs QUANT_ENGINES.md; test unitari.

Fase B — Realismo quant

- Tier-1 realism, trading friction, halt circuit; paper worker hooks.

Fase C — Scaffold exchange

- Signer, vault API, env on-chain; exchange page audit.

Fase D — RLFF

- API feedback, prompt versioning, execution guard su propose; export dataset.

16. Test, qualità e debito tecnico

16.1 Copertura attuale

- 14 file Vitest apps/web — focus motore quant e lib guard/normalize.
- 1 pytest test_strategy_engine.py — Python engine + Alpaca paths.
- Gap: zero test E2E HTTP; componenti React non testati; chat tools integration manual.

16.2 Debito noto

- Dual engine TS/Python senza unificazione codice.
- On-chain mock — non production-ready.
- yahoo-finance2 warning Node version.
- Due DB da operare (lpft + afx_dev).
- Working tree git molto dirty — molti file non committati.
- agentic-finance-exchange/ può confondere — solo archivio.

17. Operatività — avvio e troubleshooting

17.1 Avvio rapido

- Root: ./scripts/start-lpft.sh oppure npm start
- Docker: cd infra && docker compose up -d postgres redis
- Prisma: cd apps/web && npx prisma migrate deploy
- Web: cd apps/web && npm run dev → http://localhost:3000
- API: cd services/api && uvicorn lpft_api.main:app --port 8000
- Worker RQ: LPFT_REDIS_URL=... python -m lpft_worker.worker
- Intent bridge: npm run worker:intents

17.2 Problemi frequenti

- Safari non si connette: dev server :3000 spento — npm run dev.
- 503 chat: manca ANTHROPIC_API_KEY o DATABASE_URL.
- Backtest vuoto: ticker senza overlap date o Yahoo down.
- Migrazioni: usare migrate deploy su afx_dev, non lpft.

18. Variabili d'ambiente — riferimento esteso

apps/web/.env.local

- DATABASE_URL — PostgreSQL Prisma (afx_dev)
- ANTHROPIC_API_KEY — Obbligatorio per chat
- AFX_ANTHROPIC_MODEL — Modello Claude (default da SDK)
- NEXT_PUBLIC_LPFT_API_BASE — http://localhost:8000
- AFX_CHAT_DEFAULT_WALLET / NEXT_PUBLIC_AFX_DEFAULT_WALLET
- LPFT_REDIS_URL / REDIS_URL — Intent listener + event bus
- AFX_INTENTS_CHANNEL — default afx:intents:new
- AFX_CHAIN_ID, AFX_VAULT_FACTORY_ADDRESS — Scaffold chain
- AFX_SIGNER_MODE, AFX_KMS_KEY_REF, AFX_KMS_ENDPOINT — Firma
- AFX_ONCHAIN_CONFIRM_MODE — mock vs real confirmation

services/api/.env.local

- LPFT_DATABASE_URL — postgresql://lpft:lpft@localhost:5432/lpft
- LPFT_ANTHROPIC_API_KEY
- LPFT_REDIS_URL
- LPFT_AFX_INTENTS_ENABLED=true|false
- LPFT_AFX_INTENTS_CHANNEL
- LPFT_FRONTEND_BASE_URL — http://localhost:3000

19. Appendice — struttura directory sintetica

Documents/

- apps/web/ ← APP CANONICA (Next 15)
- app/ ← pages + api routes
- lib/ ← afx-*, compiler, guard
- services/quant/ ← motore TS
- prisma/ ← schema + migrations
- scripts/ ← sweep, intents, rlff export
- services/api/ ← FastAPI LPFT
- services/worker/ ← RQ consumer
- services/shared/lpft_shared/
- infra/ ← docker-compose
- docs/ ← UNIFIED_EXCHANGE, QUANT_ENGINES
- scripts/ ← start-lpft.sh, questo report PDF

20. Guardrail esecuzione e routing mercato

20.1 validateProposeExecution (afx-execution-guard.ts)

- Sharpe < -0.5 → rifiuto proposeExecution (ok: false).
- Max drawdown vs AFX_MAX_DRAWDOWN_THRESHOLD (default 35%).
- AFX_MAX_DRAWDOWN_ACTION: warn | reject | off — warn permette proposta con warning.

Test unitari in afx-execution-guard.test.ts coprono soglie e messaggi IT.

20.2 suggestMarketRoutingMode

- Crypto ticker (BTC, ETH, *-USD) → SECONDARY_AMM.
- Equity US in sessione RTH (09:30–16:00 ET, lun–ven) → PRIMARY_RFQ_ATOMIC.
- Altrimenti → SECONDARY_AMM.

isUsEquitySessionOpen usa Intl America/New_York — no dipendenza librerie esterne.

21. Motore Python lpft_shared (sintesi)

engine.py esegue programmi strategia con latenza barre, spread sintetico, onchain_latencyBars e dex_synthetic_spread_bps da ProgramMetadata. market_data.py integra fetch Yahoo/Alpaca, mock Chainlink/Pyth per costi DEX stimati. L'API espone generate-program per DSL dichiarativo; artifacts salvati per run_id.

21.1 Worker RQ

lpft_worker.worker consuma job Redis. Config allineata a LPFT_REDIS_URL e DB API. Se worker down e Redis up, API recente esegue backtest inline per evitare stallo UI legacy.

22. Script operativi (apps/web/scripts)

22.1 sweep-execution-logs.ts

npm run sweep: trova ExecutionLog in PENDING, simula conferma on-chain (mock), aggiorna transactionHash e CONFIRMED/FAILED. Usato in dev per chiudere il ciclo RLFF.

22.2 intent-listener.ts

npm run worker:intents: subscribe Redis, parse payload intent LPFT, valida router su WhitelistedDexRouter, crea ExecutionLog con idempotency, stato PENDING o LOGGED_PROPOSAL.

22.2 export-rlff-dataset.ts

npm run rlff:export: estrae ExecutionLog con feedback e metriche in formato dataset per analisi offline / training (argomenti CLI in script).

23. Compiler strategia quant (afx-quant-compiler)

parseQuantStrategyPayload accetta JSON dall'LLM. validateQuantStrategyInput verifica vincoli coerenza (simboli, pesi, risk). compileToEventDrivenConfig produce config per signal-engine (rebalance frequency, legs, asymmetric weights, halt rules). compileToEngineSpec mantiene compatibilità spec legacy per snapshot.engineSpec.

23.1 Intent class

- WALLET_MANAGEMENT — focus protezione capitale, ribilanciamento, RWA.
- ALGORITHMIC_TRADING — segnali tecnici, trend, mean reversion.

24. Stato repository Git (snapshot)

- Branch: main (tracking origin/main).
- Commit recenti: Alpaca provider; UX strategy pipeline; execution reliability; chat composer.
- Working tree: numerosi file untracked in apps/web (AFX stack), docs, migrations — dev attivo.

Raccomandazione: commit atomici per area (quant, UI chat, prisma, docs) prima di deploy condiviso.

25. Glossario

- LPFT: Laboratory / stack Python ricerca e backtest storico.
- AFX: Agentic Finance Exchange — layer exchange + chat + Prisma.
- RLFF: Reinforcement Learning from Feedback — log + feedback utente.
- RWA: Real World Asset — token sintetici tipo bAAPL.
- RFQ: Request for Quote — mercato primario fuori orario.
- T+1: Esecuzione rebalance al giorno successivo al segnale (realismo).
- HWM: High water mark — picco equity per calcolo drawdown.
- OHLCV: Open/High/Low/Close/Volume — qui prevalenza adjClose Yahoo.

26. Conclusioni e prossimi passi

Il progetto ha raggiunto un prototipo integrato end-to-end sulla UX chat e report: l'utente può conversare, generare strategie quant, vedere backtest con dati reali Yahoo, salvare strategie, aprire report analitici con registro trade, e avviare il flusso di proposta esecuzione con tracciamento RLFF. Il perimetro Python LPFT resta valido per API legacy, worker e intent bridge.

Priorità consigliate per produzione: (1) hardening on-chain e signer KMS reale; (2) test integrazione API e smoke E2E; (3) consolidamento deploy singolo (Docker/K8s); (4) decisione blockchain e whitelist router produzione; (5) riduzione debito dual-engine solo se necessario per manutenzione — altrimenti documentare confini come in QUANT_ENGINES.md.